

动态规划

数位DP

```
// 一般做法
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll num[25],n=0,dp[25][2];
ll dfs(int len,bool if6,bool shangxian){
    if(len==0) return 1;
    if(!shangxian&&dp[len][if6]) return dp[len][if6];
    ll cnt=0,maxx=(shangxian?num[len]:9);
    for(int i=0;i<=maxx;i++){
        if((if6&&i==2)||i==4){
            continue;
        }
        cnt+=dfs(len-1,i==6,shangxian&&i==maxx);
    }
    return shangxian? cnt:dp[len][if6]=cnt;
}
ll solve(ll x){
    n=0;
    memset(num, 0, sizeof(num));
    while(x){
        num[++n]=x%10;
        x/=10;
    }
    return dfs(n,false,true);
}
int main(){

    ll a,b;
    while(cin>>a>>b){
        if(a==0&&b==0) break;

        cout<<solve(b)-solve(a-1);
        cout<<endl;
    }
}
```

}

}

树形DP

```
#include<bits/stdc++.h>
// #define int long long
#define in(a) a = read_int()
using namespace std;
const int N = 2e5 + 5;
const int inf = INT_MAX;
inline int read_int() {
    int x = 0;
    char ch = getchar();
    bool f = 0;
    while('9' < ch || ch < '0') f |= ch == '-' , ch = getchar();
    while('0' <= ch && ch <= '9') x = (x << 3) + (x << 1) + ch - '0' , ch = getchar();
    return f ? -x : x;
}

struct Edge {
    int n , v , w;
} edge[N << 1];
int head[N] , eid;
void eadd(int u , int v , int w) {
    edge[++ eid].n = head[u] , edge[eid].v = v , edge[head[u] = eid].w = w;
}

int n;
int l[N] , r[N] , ans[N];
void dfs(int u , int fa) { // 树形 DP 部分
    l[u] = 1; // 初始化左边界
    for(int i = head[u] ; i ; i = edge[i].n) {
        int v = edge[i].v , w = edge[i].w;
        if(v != fa) {
            r[v] = w - 1; // 初始化右边界
            dfs(v , u);
            r[u] = min(r[u] , w - l[v]); // 转移过程
            l[u] = max(l[u] , w - r[v]);
        }
    }
}

queue<int>q;
signed main() {
    in(n);
    for(int i = 1 ; i < n ; i ++) {
        int u , v , w;
        in(u) , in(v) , in(w);
```

```

        eadd(u , v , w) , eadd(v , u , w);
    }
    r[1] = 1e9; // 初始化根节点的右边界
    dfs(1 , 0); // 求每个点权的取值范围
    for(int i = 1 ; i <= eid ; i += 2) { // 判无解部分
        int u = edge[i].v , v = edge[i + 1].v , w = edge[i].w;
        if(r[u] + r[v] < w || l[u] > r[u] || l[v] > r[v]) {
            cout<<"NO";

            return 0;
        }
    }
    ans[1] = l[1]; // 若有解，根节点点权取左边界
    q.push(1);
    while(!q.empty()) { // 广搜部分
        int u = q.front();
        q.pop();
        for(int i = head[u] ; i ; i = edge[i].n) {
            int v = edge[i].v , w = edge[i].w;
            if(ans[v]) continue;
            ans[v] = w - ans[u]; // 递推求解
        }
    }
    cout<<"YES\n"; // 打印答案
    for(int i = 1 ; i <= n ; i ++) cout<<ans[i]<<' ';
    return 0;
}

/*
#include<bits/stdc++.h>
using namespace std;
const int maxn=10000;
int cnt=1,n,head[maxn],yes[maxn],no[maxn],num[maxn],b[maxn];
struct graph{
    int next,to;
}g[maxn];
void add(int u,int v){
    g[cnt].next=head[u];
    g[cnt].to=v;
    head[u]=cnt++;
}
void dfs(int i){
    no[i]=0;
    yes[i]=num[i];

```

```

    for(int ei=head[i],v;ei;ei=g[ei].next){
        v=g[ei].to;
        dfs(v);
        no[i]+=max(yes[v],no[v]);
        yes[i]+=no[v];
    }
}

int main(){
    cin>>n;
    for(int i=1;i<=n;i++) cin>>num[i];
    for(int i=1;i<n;i++){
        int u,v;
        cin>>u>>v;
        b[u]=1;
        add(v,u);

    }
    for(int i=1;i<=n;i++){
        if(!b[i]){
            dfs(i);
            cout<<max(yes[i],no[i])<<endl;
            break;
        }
    }
}

*/

```

最长不下降子序列

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1e5+9;
int n,k,m,a[N],b[N],dp[N],t[N<<2];

void updata(int i,int l,int r,int n,int val){
    t[i]=max(val,t[i]);
    if(l==r) return;
    int mid=(l+r)>>1;
    if(n<=mid) updata(i<<1,l,mid,n,val);
    else updata(i<<1|1,mid+1,r,n,val);
    return;
}

int getmax(int i,int l,int r,int nl,int nr){
    if(l==nl&&r==nr){
        return t[i];
    }
    int mid=(l+r)>>1;
    if(mid>=nr) return getmax(i<<1,l,mid,nl,nr);
    else if(mid<nl) return getmax(i<<1|1,mid+1,r,nl,nr);
    else return max(getmax(i<<1,l,mid,nl,mid),getmax(i<<1|1,mid+1,r,mid+1,nr));
}

void solve(){
    cin>>n>>k;
    for(int i=1;i<=n;i++)cin>>a[i],b[i]=a[i];
    // 离散化
    sort(b+1,b+n+1);
    m=unique(b+1,b+n+1)-b-1;
    for(int i=1;i<=n;i++)a[i]=lower_bound(b+1,b+m+1,a[i])-b;
    for(int i=1;i<=n;i++){
        dp[i]=getmax(1,1,m,1,a[i])+1;
        updata(1,1,m,a[i],dp[i]);
    }
    //到这里dp 中存储的是以a[i]结尾的最长不下降子序列
    int ans=0;
    memset(t,0,sizeof t);
    for(int i=n;i>k;i--){
        ans=max(ans,dp[i-k]+k-1+getmax(1,1,m,a[i-k],m)+1); //通过再次利用线段树找到往后最长（题目5
        int tem=getmax(1,1,m,a[i],m)+1;
        updata(1,1,m,a[i],tem);
    }
}
```

```

    }
    ans=max(ans,k+getmax(1,1,m,a[k+1],m));
    cout<<ans<<'\n';
}

// 1 2 3 ... i-k i-k+1 i-k+2 i-k+3 ... i ...

int main(){
    ios::sync_with_stdio(false),cin.tie(nullptr),cout.tie(nullptr);
    int q=1;
    //cin>>q;
    while(q--){
        solve();
    }
    return 0;
}

```